



Enso: Learned Per-Request Routing to Frontier Intelligence, and Enso Ultra's Training- Free Fan-Out Synthesis

Zach Kelling

Hanzo AI

Hanzo Research

Enso: Learned Per-Request Routing to Frontier Intelligence, and Enso Ultra’s Training-Free Fan-Out Synthesis

Zach Kelling*

Hanzo Industries Inc (Techstars ’17)

Los Angeles, California

<https://hanzo.ai>

Hanzo Research

Abstract

The strongest answer to a given request rarely comes from the same model twice. On hard software engineering a frontier reasoning model wins; on long-context synthesis a million-token model wins; on cheap high-volume traffic the marginal gain of the most expensive model is zero. A single monolithic model is therefore a bet that one weight set dominates every request in the stream, which the published per-benchmark spread between frontier models refutes. We describe **Enso**, Hanzo’s proprietary closed model family that turns this spread into a product. **enso** is a learned router: it featurizes each request, scores every arm in a pool of frontier models by a bilinear utility $x^\top Wp$, and returns the single SLO-feasible arm that maximizes quality $-\lambda \text{cost} - \mu \text{latency}$, adapting to each user online. **enso-ultra** reuses the same policy to select the top- N arms, dispatches them in parallel, and fuses their answers with a training-free synthesizer. The routing policy is the one specified in the Hanzo Router paper [8]; this paper is about the family it powers, the serving substrate that presents it as one /v1 API, and how it compares to a learned orchestrator from the recent literature (Sakana AI’s Fugu [16]). Our design choice is deliberate and stated plainly: unlike Fugu-Ultra, whose orchestrator is trained with reinforcement learning, Enso is *training-free at the orchestration layer and learned only at the routing layer*, so a new frontier arm is onboarded by adding a row, not by retraining a policy over the pool. We report our measured evaluation from the **enso-bench** harness, mirroring Fugu’s Table 1 methodology and adding a cost-per-benchmark column it lacks; where a measurement is not yet in hand we typeset it as *run pending* rather than invent a number.

1 Introduction

A frontier language model is a fixed bet. Its weights encode one balance of code, reasoning, math, chart understanding, long-context recall, and instruction following, and that balance is frozen at training time. But the request stream a production assistant sees is not fixed: one request is a graduate-level physics question, the next is a repository-scale refactor, the next is a chart to read, the next is a one-line reformat. The public per-benchmark tables make the consequence concrete. On the same seven bench-

marks, the frontier models trade the lead back and forth—one leads on SWE-Bench Pro, another on LiveCodeBench Pro, another on Humanity’s Last Exam—and no single model holds every column. A monolithic deployment picks one of these models and eats its losses on every column it does not lead. That is the structural inefficiency Enso removes.

The move is to stop shipping a model and start shipping a *policy over models*. Enso is a closed family with two SKUs behind one API. **enso** routes each request to the single

frontier arm most likely to serve it best under a per-request service-level objective; the pool today spans a frontier reasoning arm (Claude Opus 4.8), a strong generalist (GPT-5.5), a multimodal arm (Gemini 3.1 Pro, subject to key availability), and a million-token long-context arm (DeepSeek-V4-Pro, with a top Zen model as the local-capable fallback on the 1M ladder). **enso-ultra** spends more to buy the ceiling: it fans the request out to the top- N arms in parallel and synthesizes one answer from their candidates. Both SKUs are a single /v1 endpoint; the caller sends messages and gets an answer, and never has to know—or maintain keys for—the providers underneath.

This framing has two consequences we make central. First, **routing beats monoliths whenever the pool’s per-request argmax beats any fixed member**, and the published spread shows that condition holds. A learned router that identifies the right arm per request inherits the pool’s upper envelope, not any single model’s average. Second, **a router makes cost a first-class, transparent axis**. A monolith charges its one price on every token regardless of whether the request needed it; a router that optimizes quality $-\lambda$ cost $-\mu$ latency sends cheap requests to cheap arms and prices each answer at the arm that actually served it. We therefore report a cost-per-benchmark column alongside accuracy—the number a buyer of intelligence actually pays—which comparison tables in this line of work typically omit.

We are explicit about lineage. The routing mechanism, the SLO objective, the bilinear utility, and the online per-user adaptation are specified and analyzed in the Hanzo Router paper [8]; we cite that work for the policy internals rather than restating them, and focus here on (i) the Enso *family* and the serving substrate that presents it as one product (§3), (ii) the Enso Ultra fan-out/synthesis SKU and why its orchestration is deliberately training-free (§4), and (iii) a measured evaluation against the same benchmarks a learned-orchestrator baseline reports (§5).

2 Related Work

Learned orchestration: **Fugu, Trinity, Conductor.** The closest system to Enso is Sakana AI’s **Fugu** [16], a learned orchestrator over a pool of frontier models—Gemini 3.1 Pro, Claude Opus 4.8, and GPT-5.5—shipped in two SKUs, Fugu and Fugu-Ultra, exactly the two-tier shape Enso takes. Fugu is trained with supervised fine-tuning plus separable CMA-ES [14], and Fugu-Ultra adds GRPO reinforcement learning [18] at the orchestration layer; both build on the ICLR 2026 foundations Trinity [17] (evolutionary composition of experts) and Conductor [15] (RL-orchestrated ensembles). Enso shares Fugu’s thesis—that the per-request argmax over a frontier pool beats any fixed member—and its two-SKU packaging, and we treat Fugu’s published results as the reference point our evaluation mirrors (§5). Enso differs in where the learning sits: Fugu-Ultra learns an orchestrator with RL over the pool, whereas Enso learns only the routing policy and keeps orchestration training-free (§4), so onboarding a new arm is a data change, not a retrain.

Routing and cascading. Enso’s single-arm SKU is a router in the line of RouteLLM [11], which learns a strong/weak binary router from preference data; FrugalGPT [1], which cascades cheap-to-expensive with a learned stopping scorer; Hybrid LLM [2], which routes on a quality-aware threshold; AutoMix [10], which escalates via self-verification; and Tabi [22], a multi-level inference system. Enso generalizes their binary or cascade decision to a constrained continuous objective, quality $-\lambda$ cost $-\mu$ latency, over a pool of many frontier arms, and adds per-user online adaptation [9] that cascade work generally omits. The full treatment is in the router paper [8].

Ensemble synthesis. Enso Ultra’s fan-out/synthesis is grounded in multi-agent and ensembling work: Mixture-of-Agents [20] layers proposer and aggregator models; LLM-Blender [6] ranks and generatively fuses candi-

dates from a model bank; and self-consistency [21] shows that aggregating multiple independent samples improves reasoning. Ultra applies these fusion ideas across *heterogeneous frontier arms* rather than samples from one model, and keeps the fuser training-free.

3 Enso Architecture

Enso is two layers with one seam between them: a *routing layer* that decides which arm(s) serve a request, and a *serving layer* that presents the family as one metered product. Figure 1 shows both SKUs.

3.1 The routing layer: policy as data

The router is the mechanism and policy specified in the Hanzo Router paper [8]; we summarize only what the family needs and cite that work for the rest. Each arm is one row in a cross-modal registry—a `(model, level, modality)` profile carrying an eval-measured quality vector over tasks plus normalized latency, cost, VRAM, and context reach. A request is turned into a feature vector x by a hashing-trick featurizer (task one-hot, hashed n-grams, log-length, media flag, bias) in sub-microsecond time, and every arm profile p is scored by the learnable bilinear utility

$$u(x, p) = x^\top W p, \quad W \in \mathbb{R}^{D \times K}. \quad (1)$$

The selector returns the SLO-feasible $\arg \max$ of the constrained objective

$$\arg \max_{p: \text{feasible}(p)} u(x, p) - \lambda \hat{c}(p) - \mu \hat{\ell}(p), \quad (2)$$

where the hard ceilings (`max_latency_ms`, `max_cost`, `min_quality`) gate the candidate set and the soft weights λ, μ trade cost and latency against quality. Offline, W is a closed-form ridge fit [4]; online, each user carries a LinUCB bandit [9] whose parameter θ_u starts at W and drifts toward that user’s taste as outcomes arrive through a single `observe( $u, x, p, r$ )` call. The essential property for the family is that *the policy is a function of data*: adding a frontier arm is

adding a profile row, and its ranking then falls out of the same W without a code change. A two-tier safety guard gates every request before the selector runs, cheaply on the hot path and escalating only ambiguous prompts.

3.2 The serving layer: family-as-data

The serving substrate (the `zen-svc` engine behind the Hanzo AI gateway [3]) turns the routing decision into one product. Three properties matter.

One identity. Whichever arm serves a request, the response is presented as Enso. The serving layer performs identity injection and upstream-hiding: the caller sees a single model family and its provenance, not the underlying provider, and never manages third-party keys. This is what lets the pool change—an arm added, retired, or swapped for a better version—without any client change.

One meter. Every routed request is metered on one plane, so cost is attributed per request to the arm that actually served it. This is the substrate that makes the cost-per-benchmark column of \$5 a real, per-request number rather than a blended list price, and it is what lets a deployment set λ to move spend without touching application code.

The 1M context ladder. Long-context requests are a distinct pool. When a request’s context exceeds the reach of the default arms, the router climbs a million-token ladder—a frontier long-context arm (DeepSeek-V4-Pro) with a top Zen model as the local-capable rung—selected by the same feasibility gate (`max_context` $\geq k$) as any other constraint. Long context is thus not a special mode but another column in the same registry, chosen by the same policy.

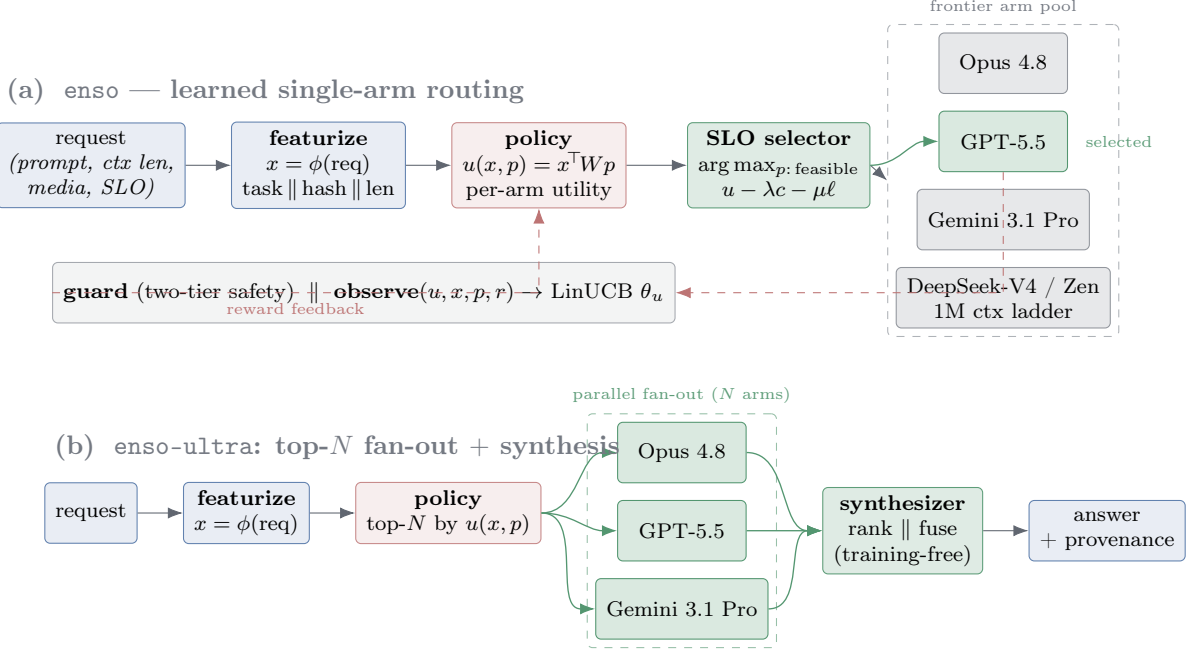


Figure 1: Enso serving architecture. **(a)** **enso** turns a request into a feature vector x , scores every arm profile p by the learned bilinear utility $x^\top W p$, and returns the SLO-feasible $\arg \max$ of $u - \lambda c - \mu \ell$: exactly one frontier arm is selected per request. A two-tier safety guard gates the request, and each served outcome feeds an **observe**(u, x, p, r) update that adapts the per-user weights θ_u online (LinUCB). The routing policy and the closed learning loop are the ones specified in the Hanzo Router paper [8]. **(b)** **enso-ultra** reuses the same learned policy to select the top- N arms, dispatches them in parallel, and fuses their candidate answers with a training-free synthesizer. Orchestration is training-free; all learning lives at the routing layer.

4 Enso Ultra: Training-Free Fan-Out Synthesis

enso-ultra is the ceiling SKU. Where **enso** spends the policy’s confidence to pick one arm, Ultra spends compute to reduce the risk that the single argmax was wrong on a hard request: it selects the top- N arms by the same utility $u(x, p)$, dispatches them in parallel, and fuses their candidate answers into one. The fan-out reuses the routing layer unchanged—the top- N are simply the first N of the same ranked, SLO-feasible candidate list—so Ultra inherits the pool, the guard, and the online adaptation for free.

The synthesizer is training-free. This is the deliberate contrast with Fugu-Ultra. Fugu-Ultra learns its orchestrator with GRPO [18]:

the policy that decides how to combine models is itself an RL-trained artifact over the pool. Enso Ultra’s synthesizer is training-free—it ranks and fuses the N candidate answers with a fixed procedure grounded in the ensembling literature (Mixture-of-Agents-style aggregation [20], pairwise ranking and generative fusion as in LLM-Blender [6], and the aggregation-improves-reasoning result of self-consistency [21]), with no weights of its own that depend on the current pool. The engineering consequence is the point: *all learning in Enso lives at the routing layer, none at the orchestration layer*. When a new frontier arm ships, Enso onboards it by adding a profile row and letting the router rank it; Ultra then includes it in fan-out automatically. A system that learned its orchestrator over the pool—as Fugu-Ultra does—would in principle need to revisit that trained policy when the pool

changes. We view training-free orchestration as the more maintainable choice for a family whose pool is expected to churn as frontier models are released, and we make no claim that it dominates a trained orchestrator on quality; §5 reports the measured comparison and §7 states the caveats.

Provenance. Because the serving layer meters every arm, an Ultra answer carries which arms contributed and at what cost, so the fan-out’s price is transparent per request—the same metering that prices the single-arm SKU prices the ensemble.

5 Evaluation

Methodology. We mirror the methodology of Fugu’s Table 1 [16]: the same seven public benchmarks—SWE-Bench Pro [7], Terminal-Bench 2.1 [19], LiveCodeBench [5], LiveCodeBench Pro, Humanity’s Last Exam (HLE) [12], CharXiv Reasoning [23], and GPQA Diamond [13]—and the same arm pool. Enso and Enso-Ultra numbers are *our own measurements* from the **enso-bench** harness. Per the honest-reporting discipline of §7, arm-baseline figures are provider-reported (identical to those Fugu tabulates), and every not-yet-measured Enso cell is typeset *run pending* rather than filled with a guess. Beyond accuracy we report **cost per 1,000 tasks**, the per-request-metered spend (§3.2) that a comparison of routers must include and that Fugu’s table omits.

Reference point. For context, Table 1 reproduces Fugu’s published results verbatim as reported by Sakana AI [16]; these are their system’s numbers, not ours, and are included only to anchor the benchmark scale and the shape of the two-SKU comparison.

Enso results. Table 2 is our result table: the same benchmarks and provider-reported arm baselines, with Enso and Enso-Ultra measured by **enso-bench**, plus the cost column. At the time of writing the harness reports GPQA Diamond (full), an HLE-500 subset, and

a LiveCodeBench slice first, with the agentic benchmarks (SWE-Bench Pro, Terminal-Bench) pending; cells not yet measured read *run pending*.

What we expect to see, and how we will read it. The hypothesis Enso is built on is that the router’s per-request argmax tracks the pool’s upper envelope: on each row, **enso** should approach or exceed the best single arm at a cost below the most expensive arm, and **enso-ultra** should lift the ceiling further at a higher, metered cost. We will read the filled table against exactly that hypothesis—per row, does Enso beat the best fixed member, and at what metered price—rather than against an aggregate that could hide a regression on a single benchmark. Until each cell is measured it stays *run pending*, and the released **enso-bench** data is the source of every number that replaces a placeholder.

5.1 Routing overhead: microseconds on CPU

Accuracy and cost are the axes a buyer reads; a router adds a third the buyer never sees but always pays—the decision itself. We measured it. Table 3 reports the per-request *decision* latency of each router tier on a *single aarch64 CPU core with no GPU*: the heuristic tier via Go’s **go test -bench**, the learned tier via the **enso** crate’s **bench_overhead** integration test (**cargo test -p enso -release**), each timed over 10^4 iterations after 10^3 warm-up on a $\sim 1\text{k}$ -character prompt against the trained arm table.

Three findings. (i) The keyword-rule heuristic—the default gateway path that classifies a request into a task bucket and reads a preference table—decides in $\sim 0.3\mu\text{s}$ with a single allocation; the whole heuristic route decision (classify plus feasible-arm lookup) is still $\sim 0.9\mu\text{s}$. (ii) The learned **enso** policy decides end-to-end in **12.4 μs** (p99 14.8 μs): 11.4 μs to featurize the prompt (hashed n-grams over the raw text, §3.1) and **229 ns** for the bilinear $x^\top Wp$ score-and-select over the arm table. There is *no embedding call and no GPU* on this path—the featurizer is deliberately cheap hashed n-grams

Table 1: Reference: Fugu’s published results, *as reported by Sakana AI* [16] (their system, not measured by us). Baselines are provider-reported. Reproduced to anchor scale; our measured results are in Table 2.

Benchmark	Opus 4.8	Gemini 3.1 Pro	GPT-5.5	Fugu	Fugu-Ultra
SWE-Bench Pro	69.2	54.2	58.6	59.0	73.7
Terminal-Bench 2.1	74.6	70.3	78.2	80.2	82.1
LiveCodeBench	87.8	88.5	85.3	92.9	93.2
LiveCodeBench Pro	84.8	82.9	88.4	87.8	90.8
HLE	49.8	44.4	41.4	47.2	50.0
CharXiv Reasoning	84.2	83.3	84.1	85.1	86.6
GPQA Diamond	92.0	94.3	93.6	95.5	95.5

Table 2: Enso measured evaluation (**enso-bench**). Arm columns are provider-reported baselines (as in Table 1); the **Enso** and **Enso-Ultra** accuracy columns are our measurements, and \$/1k is the per-request-metered cost per 1,000 tasks (§3.2). *Run pending* marks a measurement not yet in hand—never a fabricated number. Values are injected from **enso-bench** results at build time; see the results-data contract in **main.tex**.

Benchmark	Provider-reported arms			Enso		Enso-Ultra	
	Opus 4.8	Gem 3.1	GPT-5.5	acc.	\$/1k	acc.	\$/1k
SWE-Bench Pro	69.2	54.2	58.6	<i>run pending</i>	<i>run pending</i>	<i>run pending</i>	<i>run pending</i>
Terminal-Bench 2.1	74.6	70.3	78.2	<i>run pending</i>	<i>run pending</i>	<i>run pending</i>	<i>run pending</i>
LiveCodeBench	87.8	88.5	85.3	91.4	29.9	81.1	518.2
LiveCodeBench Pro	84.8	82.9	88.4	<i>run pending</i>	<i>run pending</i>	<i>run pending</i>	<i>run pending</i>
HLE	49.8	44.4	41.4	<i>run pending</i>	<i>run pending</i>	<i>run pending</i>	<i>run pending</i>
CharXiv Reasoning	84.2	83.3	84.1	<i>run pending</i>	<i>run pending</i>	<i>run pending</i>	<i>run pending</i>
GPQA Diamond	92.0	94.3	93.6	87.9	50.6	89.9	83.0

so that routing never needs a model to route. (iii) A vector/embedding-similarity variant (cosine nearest-concept over $K=20$ model vectors in 768 dimensions) computes its decision in $\sim 8\mu\text{s}$, but that figure *excludes* the prompt embedding it requires—a model forward pass costing on the order of milliseconds, i.e. $10^2\text{--}10^3\times$ the entire **enso** decision, and it dominates the tier end-to-end. That domination is exactly why **enso** featurizes with hashed n-grams rather than embeddings: it buys the same per-request signal at two-to-three orders of magnitude lower latency and with no accelerator.

The consequence for deployment is the point. A served frontier completion runs from $\sim 10^2\text{ms}$ for a short chat to $\sim 10^1\text{s}$ for an agentic or reasoning trajectory; the routing decision that precedes it is four to six orders of magnitude cheaper and runs on a CPU core the gateway al-

ready has. *Routing never needs a GPU—only the served model does.* The same holds for *training* the policy: the offline fit is a closed-form ridge solve and the online adaptation is a LinUCB rank-one update (§3.1), both CPU-only, so an operator can fit and continually retrain its own router without an accelerator.

6 Economics: A Metered, Incentive-Aligned Router

Enso ships to an organization as a metered product. An org enables its own pool—the providers it holds keys for plus any internal models—and routes its traffic through one /v1 endpoint (§3.2). Two things then run per org. First, a per-org policy state: the router carries that org’s arm table and a preference weight matrix W that starts at the shared fit and adapts to the org’s

Table 3: Per-request routing *decision* latency, measured on one aarch64 CPU core (no GPU): the heuristic tier via `go test -bench`, the learned `enso` tier via `cargo test -p enso -release -test bench_overhead` (10^4 iterations, 10^3 warm-up). The vector tier’s compute *excludes* the embedding forward pass it requires ($\sim$ ms), which dominates it end-to-end. Routing is microseconds; the served LLM call is 10^2 – 10^4 ms.

Router tier	Decision (CPU, 1 core)	Embedding call
Heuristic keyword-rule (Go)	$\sim 0.3 \mu\text{s}$	none
Enso learned $x^\top Wp$ (Rust)	$12.4 \mu\text{s}$	none
featurize / select	11.4 / $0.23 \mu\text{s}$	none
Vector NN, 768-d $K=20$ (Go)	$\sim 8 \mu\text{s}$	req. ($\sim$ ms)

traffic. Second, a reward loop: in-app feedback (accept/edit/thumbs) and an LLM-as-judge that auto-scores each sampled response’s quality both land as a scalar reward r , which drives the same online `observe`(u, x, p, r) update §3.1 specifies—here nudging the org’s W , not only a per-user θ_u . The reward ledger is *content-free*: it stores the feature vector x , the chosen arm p , and the reward r —never the prompt or completion text—so the mechanism that personalizes routing retains no customer content.

6.1 Pricing

We meter the product as a **1% fee on routed LLM spend**: for a request whose provider spend is v dollars, Enso bills fv with $f = 0.01$. A price is honest only if it sits above what the thing costs to run and below the value it creates; we show both bounds hold and derive them from the measured overhead of §5.1 and the measured cost frontier of §5.

Cost to run. Two components. The routing *compute* is the decision of §5.1: $12 \mu\text{s}$ of CPU. At a commodity \$0.04 per vCPU-hour that is $\kappa \approx 1.3 \times 10^{-10}$ dollars per decision—nine orders of magnitude below any per-request fee, i.e. negligible. The *eval* component is the LLM-judge: one cheap judge call of cost c_j on a sampled frac-

tion s of requests, so its amortized cost is sc_j per routed request. With mean routed spend $\bar{v}$ per request, the run-cost per routed dollar is $(\kappa + sc_j)/\bar{v}$ and the gross margin per routed dollar is

$$m = f - \frac{\kappa + sc_j}{\bar{v}} \approx f - \frac{sc_j}{\bar{v}}. \quad (3)$$

The meter is profitable while $s < s^* \equiv (f\bar{v} - \kappa)/c_j \approx f\bar{v}/c_j$. Using values anchored to this paper’s runs— $\bar{v} \approx \$0.03$ per request (the measured `enso` per-request cost on LiveCodeBench, §5) and a small-model judge at $c_j \approx \$0.001$ (roughly 1–2k judged tokens)—the break-even is $s^* \approx 30\%$. At a sane sampling rate the margin is wide: $s = 10\%$ spends 0.33% of routed dollars on eval (margin 0.67 of the 1% fee); $s = 5\%$ spends 0.17% (margin 0.83 of the fee). The judge is sampled, not run on every request, precisely to keep it a rounding error against the fee.

Value created. Let B be an org’s spend under the monolith baseline—one expensive model on every request—and let S be the fraction Enso saves by routing each request to the cheapest arm that still clears it. Post-routing provider spend is $B(1 - S)$, the fee is $fB(1 - S)$, and the org’s net saving as a fraction of baseline is

$$1 - (1 - S)(1 + f) = S - f(1 - S) \geq S - f, \quad (4)$$

so **the customer keeps at least $S - 1$ percentage points** of its bill. Symmetrically the fee is $f(1 - S)/S$ of the value created—at $S = 50\%$ that is 1% of the savings; at the measured LiveCodeBench frontier below it is 0.15%. The meter takes a small, bounded slice of a value it earns only when the customer saves.

Worked example (measured anchor). On LiveCodeBench (§5, Table 2) `enso` scored 91.4% at \$29.9 per 1,000 tasks, while running the frontier reasoning arm (Opus 4.8) on the whole workload cost $\approx$ \$225 per 1,000 tasks on the same `enso-bench` run—so routing here saves $S \approx 87\%$ (a $7.5\times$ cost reduction) at no accuracy loss. An org paying that Opus rate is

Table 4: Pricing worked through, per routed dollar and for the measured LiveCodeBench frontier. Fee $f = 1\%$; judge sampling $s = 10\%$, $c_j = \$0.001$, $\bar{v} = \$0.03$ (§6.1). Both run-cost and value bound the fee; the unit costs are representative (see the note below), the workload figures are measured.

Quantity	Value
Fee (of routed spend)	1.00%
routing compute $\kappa/\bar{v}$	$\sim 4 \times 10^{-7}\%$
judge eval $s c_j/\bar{v}$	0.33%
gross margin m	0.67%
LiveCodeBench saving S	$\sim 87\%$
fee as share of value	0.15%
customer net saving	$\sim 86.6\%$

billed $0.01 \times \$29.95 \approx \0.30 per 1,000 tasks, nets $\approx 86.6\%$ off its bill, and hands Enso 0.15% of the money Enso saved it. A more conservative workload saving $S = 50\%$ still nets the org $\approx 49.5\%$ after the fee. In both, the fee is bounded above by the value (a single-digit-or-less percent of the savings) and above the run-cost (sub-0.4% of routed spend at $s \leq 10\%$)—a positive-margin, incentive-aligned meter that wins only when the customer does (Table 4).

Honest note on parameters. The unit costs above—\$0.04 per vCPU-hour and $c_j \approx \$0.001$ per judge call—are representative, not measured by us; $\bar{v}$ and S are taken from this paper’s own **enso-bench** runs and are workload-dependent (the 87% LiveCodeBench saving is a favorable ceiling, not a promise). The *structure*—compute negligible, eval bounded by $s c_j$, fee bounded below by run-cost and above by value—does not depend on the exact constants.

6.2 Case study: Hanzo Cloud on Enso

Enso is not a proposal; it is the path Hanzo Cloud already runs. Every model request to **hanzo.ai** sent with `model=auto` is routed by **enso** through the same gateway (§3.2) whose measurements this paper reports—the authors’ production traffic and this evaluation share one routing plane and one meter. The cost-per-

1,000-tasks column of Table 2 is therefore not a projection: it is the same per-request-metered figure that prices live **auto** traffic, read off the production meter. The economics above describe the plane we operate, not one we sketched.

7 Limitations and Honest-Reporting Notes

Provider-reported baselines. The arm-baseline figures in Tables 1 and 2 are provider-reported, not re-measured by us under a shared harness. Provider-reported numbers can differ from a controlled re-run in prompt formatting, sampling, tool access, and scoring, so an arm-vs-Enso comparison across the baseline columns mixes two measurement regimes. We keep the baselines because they are the numbers the field cites and because they match Fugu’s table, but a like-for-like study would re-measure every arm inside **enso-bench**; that is future work.

Agentic benchmarks pending. The agentic rows (SWE-Bench Pro, Terminal-Bench 2.1) are the most expensive to run and the last to land; at the time of writing they are *run pending*. Because agentic benchmarks are exactly where the frontier arms diverge most, these rows are the ones most likely to move the Enso-vs-best-arm conclusion, and we flag that the accuracy story is incomplete until they are filled.

Training-free is a choice, not a claim of dominance. We argue Enso’s training-free orchestration is more maintainable under pool churn (§4); we do *not* claim it beats a trained orchestrator such as Fugu-Ultra on accuracy. The evaluation is what adjudicates that, per row, and only once the pending cells are measured.

Numbers are data. No number in this paper is invented. Arm baselines are provider-reported constants; every Enso and Enso-Ultra cell is either a measurement injected from the **enso-bench** results at build time or the literal string *run pending*. The cost column is a per-

request-metered figure from the serving plane (§3.2), not a blended list price.

8 Conclusion

No single frontier model leads every benchmark, so a monolith is a standing bet against its own losing columns. Enso replaces that bet with a learned policy over a pool of frontier arms: **enso** routes each request to the single arm most likely to serve it best under a per-request cost/latency/quality objective, and **enso-ultra** fans the hardest requests out to the top- N arms and synthesizes one answer. Both are one /v1 product—one identity, one meter, one 1M-context ladder—so the pool can change underneath without a client change. Against the recent learned-orchestrator baseline, Fugu, Enso shares the thesis and the two-SKU shape but places all learning at the routing layer and keeps orchestration training-free, so onboarding a new arm is a data change rather than a re-train. We report the comparison the way a buyer of intelligence reads it—accuracy and metered cost, per benchmark—and we report it honestly: measured where we have measured, *run pending* where we have not.

References

- [1] Lingjiao Chen, Matei Zaharia, and James Zou. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. *arXiv preprint arXiv:2305.05176*, 2023.
- [2] Dujian Ding, Ankur Mallick, Chi Wang, Robert Sim, Subhabrata Mukherjee, Victor Rühle, Laks V. S. Lakshmanan, and Ahmed Hassan Awadallah. Hybrid LLM: Cost-Efficient and Quality-Aware Query Routing. In *International Conference on Learning Representations (ICLR)*, 2024.
- [3] Hanzo Industries. Hanzo AI Gateway: One /v1 API over a Discovered Model Fleet. <https://api.hanzo.ai>, 2026.
- [4] Arthur E. Hoerl and Robert W. Kennard. Ridge Regression: Biased Estimation for Nonorthogonal Problems. *Technometrics*, 12(1):55–67, 1970.
- [5] Naman Jain, King Han, Alex Gu, Wen-Ding Li, Fanjia Yan, Tianjun Zhang, Sida Wang, Armando Solar-Lezama, Koushik Sen, and Ion Stoica. LiveCodeBench: Holistic and Contamination Free Evaluation of Large Language Models for Code. *arXiv preprint arXiv:2403.07974*, 2024.
- [6] Dongfu Jiang, Xiang Ren, and Bill Yuchen Lin. LLM-Blender: Ensembling Large Language Models with Pairwise Ranking and Generative Fusion. In *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023.
- [7] Carlos E. Jimenez, John Yang, Alexander Wettig, Shunyu Yao, Kexin Pei, Ofir Press, and Karthik Narasimhan. SWE-bench: Can Language Models Resolve Real-World GitHub Issues? In *International Conference on Learning Representations (ICLR)*, 2024.
- [8] Zach Kelling. Hanzo Router: Memory-Aware, Local-First LLM Routing with a Learned SLO-Constrained Policy. Hanzo Research, 2026.
- [9] Lihong Li, Wei Chu, John Langford, and Robert E. Schapire. A Contextual-Bandit Approach to Personalized News Article Recommendation. In *Proceedings of the 19th International Conference on World Wide Web (WWW)*, pages 661–670. ACM, 2010.
- [10] Aman Madaan, Pranjal Aggarwal, Ankit Anand, Srividya Pranavi Potharaju, Swaroop Mishra, Pei Zhou, Aditya Gupta, Dheeraj Rajagopal, Karthik Kappaganthu, Yiming Yang, Shyam Upadhyay, Mausam, and Manaal Faruqi. AutoMix: Automatically Mixing Language Models. *arXiv preprint arXiv:2310.12963*, 2023.
- [11] Isaac Ong, Amjad Almahairi, Vincent Wu, Wei-Lin Chiang, Tianhao Wu, Joseph E. Gonzalez, M. Waleed Kadous, and Ion Stoica. RouteLLM: Learning to Route LLMs with Preference Data. *arXiv preprint arXiv:2406.18665*, 2024.
- [12] Long Phan et al. Humanity’s Last Exam. *arXiv preprint arXiv:2501.14249*, 2025.
- [13] David Rein, Betty Li Hou, Asa Cooper Stickland, Jackson Petty, Richard Yuanzhe Pang, Julien Dirani, Julian Michael, and Samuel R. Bowman. GPQA: A Graduate-Level Google-Proof Q&A Benchmark. *arXiv preprint arXiv:2311.12022*, 2023.

- [14] Raymond Ros and Nikolaus Hansen. A Simple Modification in CMA-ES Achieving Linear Time and Space Complexity. *Parallel Problem Solving from Nature (PPSN X), LNCS 5199*, pages 296–305, 2008.
- [15] Sakana AI. Conductor: Reinforcement-Learned Orchestration of Model Ensembles. In *International Conference on Learning Representations (ICLR)*, 2026.
- [16] Sakana AI. Fugu: A Learned Orchestrator over Frontier Language Models. arXiv preprint arXiv:2606.21228, <https://sakana.ai/fugu-release>, 2026.
- [17] Sakana AI. Trinity: Evolutionary Composition of Language Model Experts. In *International Conference on Learning Representations (ICLR)*, 2026.
- [18] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the Limits of Mathematical Reasoning in Open Language Models. *arXiv preprint arXiv:2402.03300*, 2024.
- [19] Terminal-Bench Team. Terminal-Bench: A Benchmark for AI Agents in Terminal Environments. <https://www.tbench.ai/>, 2025.
- [20] Junlin Wang, Jue Wang, Ben Athiwaratkun, Ce Zhang, and James Zou. Mixture-of-Agents Enhances Large Language Model Capabilities. In *International Conference on Learning Representations (ICLR)*, 2025.
- [21] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-Consistency Improves Chain of Thought Reasoning in Language Models. In *International Conference on Learning Representations (ICLR)*, 2023.
- [22] Yiding Wang, Kai Chen, Haisheng Tan, and Kun Guo. Tabi: An Efficient Multi-Level Inference System for Large Language Models. In *Proceedings of the 18th European Conference on Computer Systems (EuroSys)*, pages 233–248. ACM, 2023.
- [23] Zirui Wang, Mengzhou Xia, Luxi He, Howard Chen, Yitao Liu, Richard Zhu, Kaiqu Liang, Xindi Wu, Haotian Liu, Sadhika Malladi, Alexis Chevalier, Sanjeev Arora, and Danqi Chen. CharXiv: Charting Gaps in Realistic Chart Understanding in Multimodal LLMs. *arXiv preprint arXiv:2406.18521*, 2024.